

腾讯云智能渗透挑战赛中AI First的学习与思考

原创 i3eg1nner SecureNexusLab 2026年5月25日 09:10 北京

“

写给自己的思考，也写给所有在 Agent 浪潮里感到既兴奋又隐隐不安的人。

”

淚笑師傅的分享和大家都不一样，他没有急着展示工具，而是先提了一个问题：「**渗透测试的本质是什么？**」

传统答案是信息收集。但他说，信息收集只是一个动作，执行完之后已知变多了，然后呢？从更高维度看，渗透测试是「**无限状态空间中的有向搜索**」——起点已知，终点已知（拿 shell、得 flag），但中间的路径是未知的。信息收集，不过是在这片未知空间里移动的方式之一。

将问题抽象成更高维度，让我一开始就感受到了淚笑師傅思考的核心。因为一旦你这样看问题，就会发现：数学证明也是这个结构，漏洞挖掘也是，很多工程难题也是。如果你设计出一套能在这类结构中工作的系统，你就有了「**通用问题求解引擎**」的雏形。

他把这套系统叫做「**Cairn AI**」。



核心：黑板架构，以及它背后的设计哲学

黑板架构不是新概念——它在 1970 年代的语音识别系统里就被提出了。淚笑師傅说，他独立推导出这个模式之后，才查阅资料发现它已经有名字了。

这件事本身就很有意思：某些底层结构，当你真正想清楚问题的时候，会自然地收敛到同一个形状。

类比是这样的：一群侦探围坐在一块黑板前破案。

- 「Fact（事实）」：黑板上已经写下的已确认信息，是图的节点。
- 「Intent（意图）」：接下来要做什么，是图的边，未执行时是问号，执行完之后变成新的 Fact。
- 「Hint（提示）」：来自图外的高维输入——人类的洞见、题目本身的线索。它不参与图内推理，但作为外部参考注入系统。



整个推进过程是一个循环，对应军事理论里的 OODA：

```
1 ounter(line
2  Observe (观察) → Orient (分析) → Decide (决策) → Act (执行)
```

对应到系统里三种任务：

1. 「Bootstrap」：先直接尝试解决问题，大概率失败，但把尝试本身和发现的信息写回图里。
2. 「Reason」：读取整张图的当前状态，推理下一步应该做什么，产出新的 Intent。
3. 「Explore」：执行 Intent，把结果写成新的 Fact。

就这三件事，循环往复，直到某条边直接连到了终点。



AI First 的核心理念

浪笑师傅在演讲里强调了一句话，我认为这是整个系统中，思考问题的逻辑变化后的最直观体现：

「整个过程中，没有任何一行代码告诉 Agent 渗透测试应该怎么做。」

Agent 看到的只有当前的图，和本次要执行的任务类型。分工从哪里来？从图里来，从运行时的态势里来——新的 Fact 写入，触发新一轮 Reason，图上长出新的 Intent，Worker 去 Explore。



对比传统的多 Agent 架构：信息收集 Agent、漏洞利用 Agent、报告生成 Agent……这种预定义角色的分工，是人类在设计时强加进去的。它看上去结构清晰，但本质上是「把人类对问题的理解硬编码进了架构」。

问题是：人类对问题的理解，往往是残缺的。传统多 Agent 架构是人类局限的投影，分工带来信息孤岛，上下文残缺，翻译和过滤损耗。它看似在设计智能，实则是在复刻人类的局限。

Cairn AI选择的路径是：「**找到状态空间的最小表达。**」从蚁群算法看，蚂蚁不相互点对点通话，它们只在路上留下信息素，更好的路径信息素浓度更高，群体行为从个体的简单规则里涌现出来。Cairn AI的 Worker 也一样：相互之间绝不通信，只通过写 Fact 和 Intent 间接协调，整体的搜索策略从这个简单循环中涌现。

「**Less is More，去掉一个东西，比加一个东西需要更深的理解。**」



在 DeepSeek 屠龙刀的背景下谈这件事

DeepSeek 的出现，把推理成本打到了一个新的地板。这不是量变，是质变——它意味着：原本因为成本原因不值得用 LLM 来做的事情，现在可以放手去做了。像 Cairn AI 这样的系统，每一轮循环都要调用模型来 Reason 和 Explore，在 Claude opus 时代的定价下，跑一套渗透的成本是不可忽视的。现在，这个约束在松动。

这是好事。它让更多人能够真正验证这些架构思路，而不只是在 demo 里点到为止。但它也在加速我对网安未来和个人发展的焦虑。

过去我们谈技术债，谈的是人类写的烂代码：注释缺失、耦合严重、没有测试、逻辑混乱，堆了十年的屎山，没有人敢动，也没有人真正理解。

现在，有了 Agent，有了 Claude Code，有了各种 AI 编程工具，开发速度上去了，代码量暴增，demo 一个接一个。我在一些团队里观察到一个现象：「**系统演进的速度，开始超过人类理解它的速度。**」

这不是比喻，一个由 Agent 驱动的系统，在快速迭代中，某个模块是怎么工作的，为什么这样工作，某个边界条件下会发生什么——开发者自己开始说不清楚了。因为那段逻辑是 AI 生成的，通过了测试，看上去没问题，就合进去了。

在 demo 迭代和验证场景里，这是可以接受的代价，快速验证思路，跑通流程，这是 AI 辅助开发的合理用法。

旧的屎山，是技术债，而新的屎山，是黑箱债。AI 生成的代码，行为符合测试用例，但背后的逻辑链条对于写它的人来说是不透明的。当出了问题，你连这里为什么这样写都无法追问，因为根本没有人做过这个决策。

「技术债是人类的代码质量留下的。黑箱债是人类的理解速度跟不上 AI 的生产速度留下的。」

这两者的风险属性完全不同。技术债积累到一定程度，会让开发变慢，最终迫使重构。黑箱债积累到一定程度，会在某个不可预期的角落出现某个不可解释的行为，而你没有任何的上下文去理解它为什么会发生。

Cairn AI的设计，恰好是这个问题的反面

有趣的是，淚笑师傅的系统设计，恰好是在对抗这种趋势。

黑板架构的核心特征是：**「全局状态对所有参与者透明」**。每一个 Fact，每一个 Intent，都写在黑板上，任何时刻你都可以看到整张图，知道系统走到了哪里，下一步在想什么。

这不只是架构选择，这是一种**「可解释性原则」**。系统不在各个 Agent 的私有状态里藏东西，所有的中间过程都外化成图上的节点和边。

这和现在很多 Agent 系统的做法相反。很多系统把复杂性藏在 Agent 的内部状态里，表面上看起来封装得很好，实际上是把黑箱往更深处推。淚笑师傅说，他们的系统没有用任何预设的渗透工具流程，没有角色定义，没有硬编码的知识。但我认为，比这更重要的是：**「系统的推理过程是可观测的」**。你可以看着图一步一步长大，看着 Intent 被执行变成 Fact，理解系统在做什么，为什么这样做。

这才是在 Agent 时代真正有价值的工程属性。

理解深度，是这个时代真正稀缺的东西

淚笑师傅最后说了一句话，我觉得可以作为这篇笔记的结尾：



「去掉一个东西，比加一个东西需要更深的理解。」

”

在 AI 工具满天飞的现在，这句话更难做到，也更值钱。DeepSeek 给了我们更便宜的推理，Claude Code 给了我们更快的开发速度，各种 Agent 框架给了我们更多的工具。但这些都是放大器——它们放大你已有的理解，也放大你理解的空缺。

Cairn AI对我最大的启发是：在你开始堆工具之前，先想清楚你要解决的问题的本质结构是什么，找到最小表达，再开始构建。

这不是 AI-First，这是 **「Think-First」**。

AI，只是这之后的事情。

Harness VS AI First

这次智能渗透挑战赛里，Harness关键词的出场频率很高，因此学习和复盘的时候我也在思考 Harness 和 AI First的区别，这里也简单分享下我的观点。相比于AI First，其实Harness更为大家熟知，Harness的编排给 LLM 套上一个脚手架，定义了相对稳定的运行过程。现在市面上大多数 AI 渗透工具走的是这条路，让 LLM 按照某种流程编排调用。

Harness 的价值在于 **「可控性和可解释性」**。当你的任务是可控和可维护，而非直白地完成目标，Harness 的预定义边界是优势，这种约束的明显应用在于公司代码开发，前面提到的 AI 代码技术债，Harness就是目前大家较为认可的缓解方式。通过Harness的设计，起码保证了业务代码经过良好地测试，不一定是最优解，但一定是维护性较好、缺陷较少的。

所以，更准确地说，Harness 和 AI First 有各自的舒适区，渗透任务的特殊性使得AI First的理念在完成任务上表现更好，套用和淚笑师傅交流时的一句话：**「设计agent系统，本质上是在“控制论”与“涌现”之间找平衡。」**

最近新建了微信技术交流群，欢迎各位师傅添加小助手微信拉你进群：



相关阅读推荐：

Oxidizer：发表在顶会上的 Rust 反编译工具

内卷的风还是吹到了漏洞利用：ExploitGym 全自动漏洞利用Agent评估

